Manual del Programador — PiedraPapelTijera

Proyecto: `piedrapapelotijera` — Actividad 7 (LPR 5.º 3.ª) **Versión:** 1.0.0 **Autor:** Ian Quiroga **Fecha:** 2026

---

1. Descripción del sistema

Juego de Piedra, Papel o Tijera contra la IA de consola, con **persistencia en disco**: cada partida se registra en `src/historial_partidas.txt` y puede recuperarse en ejecuciones posteriores.

2. Arquitectura del sistema

```
main()
 ■■■ srand(time(0))              semilla de la IA
 ■■■ do { ... } while (op != 3)  menú principal
 ■    ■■■ op == 1 ■■ jugarPartida(&contadorID)
 ■    ■                ■■■ getline del nombre
 ■    ■                ■■■ rand() % 3 para la IA
 ■    ■                ■■■ reglas Victoria / Empate / Derrota
 ■    ■                ■■■ guardarEnDisco(p)  ■■ ofstream (ios::app)
 ■    ■■■ op == 2 ■■ leerHistorialDisco()     ■■ ifstream + getline(',')
 ■    ■■■ op == 3 ■■ salir
 ■■■ return 0
```

3. Modelo de datos

```
struct Partida
{
    int    idPartida;
    string jugador;
    string eleccionJugador;   // "Piedra" | "Papel" | "Tijera"
    string eleccionIA;
    string resultado;         // "Victoria" | "Empate" | "Derrota"
};
```

4. Especificación de las funciones

4.1 `void jugarPartida(int& contadorID)`

• **Parámetro:** `contadorID` — se pasa **por referencia** para incrementar el

identificador de la partida sin perder el valor actualizado en `main()`.

• **Secuencia:** nombre → opción del jugador (1-3) → `rand() % 3 + 1` para la IA →

comparación de reglas → `guardarEnDisco(p)`.

Reglas implementadas:

Jugador	IA	Resultado
Piedra	Tijera	Victoria
Papel	Piedra	Victoria
Tijera	Papel	Victoria
Igual	Igual	Empate
Cualquier otro	—	Derrota

4.2 `void guardarEnDisco(const Partida& p)`

• **Parámetro:** `const Partida&` — paso por **referencia constante**: la función

lee la estructura sin copiarla y no puede modificarla.

• **Abre:** `ofstream archivo("src/historial_partidas.txt", ios::app);`

• **Formato:** CSV → `id,jugador,eleccionJugador,eleccionIA,resultado`

• **Verifica:** `archivo.is_open()`; si falla, informa por `cerr` y retorna.

> ■■ `ios::app` **es obligatorio.** Sin él, el modo por defecto (`ios::out`) > trunca el archivo en cada escritura y se pierde todo el historial anterior.

4.3 `void leerHistorialDisco()`

• **Abre:** `ifstream archivo("src/historial_partidas.txt");`

• **Lee** con `getline(archivo, campo, ',')` encadenados mediante cortocircuito

(`&&`), lo que garantiza que solo se procese una línea completa.

• **Cuenta** los registros en `total` e informa el total leído.

5. Rutas relativas: punto crítico

Ambas funciones usan la ruta `src/historial_partidas.txt`, que es **relativa al directorio de trabajo actual**. El ejecutable debe lanzarse **desde la raíz de la carpeta** `piedrapapelotijera/`:

```
cd .\piedrapapelotijera
g++ -std=c++11 src/main.cpp -o src/piedrapapelotijera.exe
.\src\piedrapapelotijera.exe
```

Si se ejecuta con doble clic desde el explorador, el directorio de trabajo será `src/` y el archivo se creará en `src/src/historial_partidas.txt`. Para evitarlo, conviene copiar el `.exe` a la raíz o usar `chdir()` de `<direct.h>`.

6. Compilación

```
g++ -std=c++11 src/main.cpp -o src/piedrapapelotijera.exe
```

El flag `-std=c++11` es necesario para `<cstdlib>`, `<ctime>` y `<fstream>` en compiladores antiguos (GCC 4.x). Con GCC 8 o superior funciona sin el flag.

7. Errores conocidos y soluciones

Síntoma Causa Solución
`Aun no existen partidas guardadas en disco` No se jugó ninguna partida todavía Elegir la Opción 1 primero
El archivo no aparece en `capturas/` Se ejecutó desde otro directorio Lanzar desde la raíz de la actividad
El nombre se lee vacío Falta `cin.ignore()` Ya está en el código; no eliminarlo
Solo se ve la última partida Se quitó `ios::app` Restaurar el flag en `ofstream`
Caracteres `?` en el `.txt` Codificación de consola distinta Abrir el `.txt` con Bloc de notas en UTF-8

8. Extensiones posibles

- • Guardar fecha y hora con `time_t` / `strftime`.
- • Estadísticas: victorias, derrotas y empates leídas del archivo.
- • Interfaz gráfica conraylib o SFML.
- • Exportar el historial a un `.csv` y abrirlo en una hoja de cálculo.

9. Control de versiones

Versión Descripción
`v1.0.0` Juego completo con persistencia en disco mediante `struct` + `fstream`.